IOS 低功耗 API

扫码二维码区分设备类型

{"ACT":"Add","ID":"VSTK000001AAAAA","DT":"BMG1","WiFi":"20"}
WiFi："20"　代表设备是 4G 低功耗设备，需要调 4G 接口
WiFi："18"　代表设备是 wifi 低功耗设备，需要调 wifi 接口（备注：需要注意 DT 确定机型，ipc 也会使用）

1.- (int)MagLowpowerDeviceConnect:(NSString*) strIP;*指定低功耗服务器 IP 地址，返回 1 代表成功，其它代表是失败的。（在 MagLowpowerInitDevice 前一定要调用此接口）*

例如——————————————————————————————

```
- (NSString *)GetIPbyName
{
    NSLog(@"GetIPbyName----start");
    Boolean result,bResolved;
    CFHostRef hostRef;
    CFArrayRef addresses = NULL;
    CFStringRef hostNameRef = CFStringCreateWithCString(kCFAllocatorDefault, "liteos-master.eye4.cn",kCFStringEncodingASCII);
    hostRef = CFHostCreateWithName(kCFAllocatorDefault, hostNameRef);
    if (hostRef) {
        result = CFHostStartInfoResolution(hostRef, kCFHostAddresses, NULL);
        if (result == TRUE) {
            addresses = CFHostGetAddressing(hostRef, &result);
        }
    }
    bResolved = result == TRUE ? true : false;
    NSString *strIp;
    if(bResolved)
    {
        struct sockaddr_in* remoteAddr;
        for(int i = 0; i < CFArrayGetCount(addresses); i++)
        {
            CFDataRef saData = (CFDataRef)CFArrayGetValueAtIndex(addresses, i);
            remoteAddr = (struct sockaddr_in*)CFDataGetBytePtr(saData);
            if(remoteAddr != NULL)
            {
                if (remoteAddr->sin_family == AF_INET6) {
                    struct sockaddr_in6 *ip6 = (struct sockaddr_in6*)CFDataGetBytePtr(saData);
                    char str[INET6_ADDRSTRLEN]={0};
                    const char* szRet = inet_ntop(AF_INET6, &(ip6->sin6_addr), str, sizeof(str));
                    if (szRet != NULL && strlen(str) > 0) {
                        strIp = [NSString stringWithUTF8String:str];
                        [[VSNet shareinstance] SetMagLowpowerSocketIPV6];
                    }
                    NSLog(@"AF_INET6");
                }
                else if (remoteAddr->sin_family == AF_INET)
                {
                    char str[INET_ADDRSTRLEN] = {0};
                    const char* szRet = inet_ntop(AF_INET, &(remoteAddr->sin_addr), str, sizeof(str));
                    if (szRet != NULL && strlen(str) > 0) {
                        strIp = [NSString stringWithUTF8String:str];
                    }
                    NSLog(@"AF_INET");
                }
                else
                    NSLog(@"AF_INET Undefined family.");
            }
        }
    }
    CFRelease(hostNameRef);
    CFRelease(hostRef);
    NSLog(@"GetIPbyName----stop");
    return strIp;
}
```

```
    __weak AppDelegate *weakSelf = self;
    dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_BACKGROUND, 0),^{
        weakSelf.DB1Ip = [weakSelf GetIPbyName];
        [[VSNet shareinstance] MagLowpowerDeviceConnect:weakSelf.DB1Ip];
    });
```

2.– (void)MagLowpowerDeviceDisconnect;*断开低功耗服务器的连接。*

3.– (int)MagLowpowerInitDevice:(NSString *)deviceIdentity;*初始化注册低功耗设备，返回 1 代表成功，其它代表是失败的。（如果是失败的会一直尝试重新连接直到连接注册上为止，退出可使用* MagLowpowerDisconnetDevice 接口）

例如——————————————————————————————

```
if (self.firmwareVersion == LOWPOWER_FIRMWARE_VERSION || [[PublicDefine getDevModelWithUID:_devId]
    [PublicDefine getDeviceModeWithUid:_devId] == Camera_S1) {
    [[VSNet shareinstance] setLowpowerDeviceDelegate:self];
    [[VSNet shareinstance] MagLowpowerInitDevice:self.devId];
    [[VSNet shareinstance] MagLowpowerAwakenDevice:self.devId];
    [[VSNet shareinstance] MagLowpowerKeepDeviceActive:self.devId Time:30];
```

4.– (int)MagLowpowerDisconnetDevice:(NSString *)deviceIdentity;断开设备

5.– (int)MagLowpowerAwakenDevice:(NSString *)deviceIdentity；*唤醒设备返回 1 代表成功，其它代表是失败的。*

6.– (int)MagLowpowerGetDeviceStatus:(NSString *)deviceIdentity *向服务器查询低功耗设备状态*

7.–(int)MagLowpowerKeepDeviceActive:(NSString *)deviceIdentity Time:(int) time

保活设备通讯，此接口用于 P2P 连接上后保活设备不让其睡眠，返回 1 代表成功，其它代表是失败的(time 设备休眠倒计时单位为秒，此接口调用一次即可，多次调用以最后一次调用传的 time 为准）

8.–(int)MagLowpowerRemoveKeepDeviceActive:(NSString *)deviceIdentity;*移除保活设备通讯(删除之前的设置保活通讯，设备休眠倒计时到了就会休眠）*

9.– (void)setLowpowerDeviceDelegate: (id <LowpowerDeviceProtocol>) delegate

设置低功耗设备代理，用于接收设备状态

10.–(void)SetMagLowpowerSocketIPV6；当前手机使用的是 IPV6 网络

二、接收设备状态

```
enum EM_LOWPOWER_NOTIFY_STATUS
{
    EM_LOWPOWER_NOTIFY_AGAIN_P2PSTART    = -3,//需要重新调用 Start p2p 接口
    EM_LOWPOWER_NOTIFY_AGAIN_INITDEVICE = -2,//需要重新调用 MagLowpowerInitDevice

    EM_LOWPOWER_NOTIFY_REGING          = 8,   //正在注册中，由于 MagLowpowerGetDeviceStatus 接口才能取到
    EM_LOWPOWER_NOTIFY_Unknown         = 9,   //错误无法识别的状态

    EM_LOWPOWER_NOTIFY_ONLINE            = 10,  //在线
    EM_LOWPOWER_NOTIFY_OFFLINE           = 11,  //离线
    EM_LOWPOWER_NOTIFY_SUPER_SLEEP     = 13,  //超低功耗休眠
```

```
    EM_LOWPOWER_NOTIFY_SLEEP                = 22,  //休眠

    EM_LOWPOWER_NOTIFY_SET_ONLINE       = 30, //P2P 已经连接上(p2p 在线)
    EM_LOWPOWER_NOTIFY_P2PCONNETING     = 40,  //P2P 连接中
};
```

例如：

```
-(void) DeviceStateNotify:(NSString*)strUID state:(int) nState {
    if (![strUID isEqualToString:_devId]) {
        return;
    }
    weakSelf(weakSelf);
    dispatch_async(dispatch_get_main_queue(), ^{
        [weakSelf refreshLowpowerStatus:strUID state:nState];
    });
}

- (void)refreshLowpowerStatus:(NSString*)strUID state:(int) nState {
    [mAppDelegate.cameraListManagement refreshLowpowerDevStatus:strUID withStatus:nState];
    NSDictionary *dic = [mAppDelegate.cameraListManagement GetCameraAtDID:self.devId];
    if (nState == LOWPOWER_STATUS_ONLINE) {
        if ([strUID isEqualToString:_devId]) {
            NSLog(@"门铃在线start");
            if([[VSNet shareinstance] GetP2PConnetState:strUID] ==EM_GETP2PCONNET_STATE_NOTINIT){
                NSString *pwd = dic[@STR_PWD];
                [self startLowpowerDev:strUID pwd:pwd];
            }
        }
    }
    else if (nState == EM_LOWPOWER_NOTIFY_AGAIN_P2PSTART) {
        if([[VSNet shareinstance] GetP2PConnetState:strUID] ==EM_GETP2PCONNET_STATE_NOTINIT){
            NSString *pwd = dic[@STR_PWD];
            [self startLowpowerDev:strUID pwd:pwd];
        }
    }
    else if (nState == EM_LOWPOWER_NOTIFY_AGAIN_INITDEVICE) {
        [[VSNet shareinstance] setLowpowerDeviceDelegate:self];
        [[VSNet shareinstance] MagLowpowerInitDevice:self.devId];
        [[VSNet shareinstance] MagLowpowerAwakenDevice:self.devId];
        [[VSNet shareinstance] MagLowpowerKeepDeviceActive:self.devId Time:30];
    }
    else if (nState == LOWPOWER_STATUS_SLEEP || nState == LOWPOWER_AutoSTATUS_SLEEP) {
        if ([strUID isEqualToString:_devId]) {
            NSLog(@"报警门铃睡眠:%d",[dic[@STR_PPPP_STATUS] intValue]);
            [mAppDelegate.cameraListManagement UpdatePPPPStatus:strUID status:PPPP_STATUS_CONNECTING];
            [[VSNet shareinstance] setLowpowerDeviceDelegate:self];
            [[VSNet shareinstance] MagLowpowerAwakenDevice:strUID];
            [[VSNet shareinstance] MagLowpowerKeepDeviceActive:strUID Time:30];
            _ppppStatus = (int)PPPP_STATUS_CONNECTING;
            __weak ParameterSettingViewController *weakSelf = self;
            dispatch_async(dispatch_get_main_queue(), ^{
                weakSelf.navigationItem.rightBarButtonItems = nil;
                weakSelf.firstDataSource = nil;
                [self->_tableview reloadData];
                [[NSNotificationCenter defaultCenter] postNotificationName:@"CameraNotOnLine" object:nil];

            });
        }
    }
    _lowerState = nState;
}
```

说明文档的接口是 wifi 版本低功耗设备的 API。4G 网络设备的 API 如下：

```
#pragma mark/*************************4g设备端接口beg************************************/
//置前台需要连接服务器
- (int)MagFlowDeviceConnect:(NSString*) strIP;
//置后台需要断开服务器
- (void)MagFlowDeviceDisconnect;

- (void)setFlowDeviceDelegate: (id <LowpowerDeviceProtocol>) delegate;

- (int)GetFlowServerConnectStatus;

//初化设备
- (int)MagFlowInitDevice:(NSString *)deviceIdentity;
//取设备状态
- (int)MagFlowGetDeviceStatus:(NSString *)deviceIdentity;
//唤醒设备
- (int)MagFlowAwakenDevice:(NSString *)deviceIdentity;
//断开设备
- (int)MagFlowDisconnetDevice:(NSString *)deviceIdentity;

//是IPV6的网络
-(void)SetMagFlowSocketIPV6;

/**
 *  保持设备激活
 *  @param deviceIdentity 设备id
 *  @param time   设备延时休眠时间不得少5秒
 */
-(int)MagFlowKeepDeviceActive:(NSString *)deviceIdentity Time:(int) time;

/**
 *  移除保持设备激活（与MagLowpowerKeepDeviceActive成对的）
 *  @param deviceIdentity 设备id
 */
-(int)MagFlowRemoveKeepDeviceActive:(NSString *)deviceIdentity;
/**
 *  指定设备休眠倒计时
 *  @param deviceIdentity 设备id
 *  @param time   设备延时休眠时间
 */
-(int)MagFlowSleepDevice:(NSString *)deviceIdentity Time:(int) time;

#pragma mark/*************************4g设备端接口end************************************/
```